

C++ Placement Prep — Day 1

Interview Q&A — Classes, Objects, OOP Basics

These are the questions an interviewer is likely to actually ask out loud on this topic, each answered in the structure to use when speaking: **Definition** → **Why it matters** → **Example** → **Likely follow-up**.

Read each answer, then close the PDF and try saying it out loud in your own words. That's what makes it usable in the actual interview, not just recognizable on a page.

Q1. What is the difference between a class and a struct in C++?

DEFINITION

Both let you group data and functions together. The only functional difference is the default access level — struct members are public by default, class members are private by default.

WHY IT MATTERS

It shows you understand C++ didn't reinvent struct, it extended it. This is usually an easy opening question, so answer it confidently and briefly instead of over-explaining.

EXAMPLE

```
struct Point { int x, y; };      // x, y are public
class Point { int x, y; };      // x, y are private
```

Follow-up to expect: "So can a struct have constructors and member functions too?" — Yes, in C++ (unlike C) it can, unlike in plain C where struct is data-only.

Q2. What is encapsulation, and why does it matter?

DEFINITION

Encapsulation means bundling data and the functions that operate on it into one unit (a class), and restricting direct access to that data using access specifiers.

WHY IT MATTERS

It prevents an object from being put into an invalid state from outside code, and it lets you change the internal implementation later without breaking code that uses the class.

EXAMPLE

```
class BankAccount {
    private: double balance;
    public:
        void withdraw(double amt) {
            if (amt <= balance) balance -= amt;    // guarded
        }
};
```

Follow-up to expect: "Why not just make balance public and check the condition in main()?" — Because then every piece of code that touches balance has to remember to check the condition; encapsulation makes that impossible to forget.

Q3. What is a constructor, and why do we need one?

DEFINITION

A constructor is a special member function with the same name as the class, no return type, called automatically the moment an object is created.

WHY IT MATTERS

It guarantees an object starts in a valid, fully-initialized state — you can't forget to call it, unlike a regular `init()` function which someone could forget to call.

EXAMPLE

```
class Rectangle {
    public:
        double length, width;
        Rectangle(double l, double w) : length(l), width(w) {}
};
Rectangle r(4.0, 6.0); // constructor runs automatically here
```

Follow-up to expect: "What if you don't write any constructor at all?" — The compiler generates an empty default constructor for you automatically.

Q4. What is constructor overloading?

DEFINITION

Defining multiple constructors in the same class with different parameter lists, so objects can be created in different ways.

WHY IT MATTERS

Same reasoning as function overloading generally — flexibility at the call site, the compiler picks the right version based on the arguments you pass.

EXAMPLE

```
Rectangle() : length(0), width(0) {} // default
Rectangle(double l, double w) : length(l), width(w) {} // parameterized
```

Follow-up to expect: "What happens if you define a parameterized constructor but no default one, and then write `Rectangle r;`?" — Compile error. Defining any parameterized constructor removes the compiler's free default constructor.

Q5. What is a destructor, and when does it run?

DEFINITION

A special member function named `~ClassName()`, no return type, no parameters, called automatically when an object goes out of scope or is explicitly deleted.

WHY IT MATTERS

It's where you release resources the object was holding — dynamically allocated memory, open file handles — so you don't leak them.

EXAMPLE

```
class Buffer {
    double *data;
    public:
        Buffer() { data = new double[100]; }
        ~Buffer() { delete[] data; } // cleanup
};
```

Follow-up to expect: "Can a class have two destructors?" — No, destructors take no arguments at all, so there is nothing to overload on.

Q6. What is the 'this' pointer?

DEFINITION

Inside any non-static member function, 'this' is a hidden pointer to the exact object the function was called on.

WHY IT MATTERS

Most commonly used to resolve a naming conflict when a parameter has the same name as a member variable.

EXAMPLE

```
void setLength(double length) {  
    this->length = length;    // this->length is the member,  
}  
                             // length (no this->) is the parameter
```

Follow-up to expect: "Can 'this' be used in a static member function?" — No, static functions don't belong to any specific object, so there's no 'this' to point to.

Q7. What is the difference between a reference and a pointer?

DEFINITION

A reference is an alias for an existing variable — another name for the same memory. A pointer is a separate variable that stores an address.

WHY IT MATTERS

References must be initialized when declared and can never be reseated to refer to something else, which makes them safer for simple pass-by-reference use than pointers.

EXAMPLE

```
int x = 10;  
int &r = x;    // r IS x — cannot later do r = &y;  
int *ptr = &x; // ptr holds the address of x — can be reassigned
```

Follow-up to expect: "Can a reference be null?" — No, and that's actually one of its main safety advantages over a pointer.

Q8. Why use access specifiers like private instead of making everything public?

DEFINITION

Access specifiers control which code is allowed to touch a class's internal data directly.

WHY IT MATTERS

Making everything public defeats the point of a class — any code anywhere could set an object into an invalid state, and there'd be no single place to add validation later.

EXAMPLE

```
class Circle {  
    private: double radius;    // can't go negative from outside  
    public:  
        void setRadius(double r) { if (r >= 0) radius = r; }  
};
```

Follow-up to expect: "What's the difference between private and protected?" — protected additionally allows access from derived (child) classes, private does not, even for children.

Q9. What is the scope resolution operator (::) used for?

DEFINITION

It's used to define a class's member function outside the class body, linking the definition back to its declaration.

WHY IT MATTERS

It lets you separate a class's interface (in a header) from its implementation (in a .cpp file) — same reasoning as splitting .h and .c files in embedded C.

EXAMPLE

```
class Rectangle {  
    public: double getArea();    // declared here  
};  
double Rectangle::getArea() { return length * width; } // defined here
```

Follow-up to expect: "Is this required, or just convention?" — Not required for small classes, but standard practice for anything beyond a toy example.

Q10. What is an initializer list, and why prefer it over assigning in the constructor body?

DEFINITION

Syntax that initializes member variables directly as part of construction, written after a colon following the constructor's parameter list.

WHY IT MATTERS

It's slightly more efficient (initializes directly instead of default-constructing then reassigning), and it's the only way to initialize const members or reference members.

EXAMPLE

```
Rectangle(double l, double w) : length(l), width(w) {  
    // body can be empty  
}
```

Follow-up to expect: "Give an example where you're forced to use it." — Any class with a const member or a reference member must use an initializer list; assignment in the body won't compile for those.

Practice tip: cover the answer, read only the question, and speak your answer out loud in the Definition → Why it matters → Example order before checking. That's the exact structure an interviewer is listening for.